

week3

1~2주차는 짧게 훑기

1주차: K8s CR/CRD & Go 환경

CRD (CustomResourceDefinition)

새로운 리소스 종류를 쿠버네티스 API 서버에 등록하는 정의. "이런 필드를 가진 Memo 라는 리소스 종류가 있다"를 선언하는 스키마.

쿠버네티스 API 서버를 확장하는 가장 가벼운 방법입니다. Pod, Deployment 처럼 쿠버네티스가 기본 제공하는 리소스 외에 내가 정의한 리소스 종류

CR (Custom Resource)

그 CRD를 기반으로 만들어진 실제 인스턴스. `kubectl apply -f memo.yaml` 로 생성하는 개별 객체.

2주차: Controller 패턴 & client-go

Reconcile의 본질

쿠버네티스의 모든 컨트롤러는 단 하나의 루프를 돈다.

```
for {
    desired := getDesiredState() //etcd에 저장된 spec
    current := getCurrentState() //실제 클러스터 상태
    if desired != current {
        takeAction() // 차이를 줄이는 행동
    }
}
```

⇒ 선언형(declarative) 설계의 핵심!

"Pod 3개를 만들어라" X → "Pod가 3개 있어야한다" 라고 선언하면, 컨트롤러가 그 상태를 유지합니다. 누가 Pod를 죽이면 다시 만들고, 노드가 죽으면 다른 노드에 다시 띄우고, Argo Rollouts 트러블 슈팅할 때 만난 `progressDeadlineSeconds` 도 결국 "이 시간안에 desired 에 도달 못하면 abort" 라는 reconcile 규칙

(reconcile 규칙? 원하는 상태(desired state)와 현재 상태(current state)를 일치시키는 컨트롤러 루프 동작)

Watch vs Polling

```
kubectl get pods -w
```

컨트롤러는 API 서버에 long-lived HTTP 연결을 걸어두고 변경 이벤트(ADDED/MODIFIED/DELETED)를 푸시 받는다. → 이게 etcd의 watch 메커니즘과 직결

⇒ API 서버가 etcd watch를 클라이언트에게 중계한다

client-go 3대 컴포넌트

- Informer: API 서버에서 Watch 이벤트를 받아오는 컴포넌트.
 - 받아와서 하는일 (1) 로컬 캐시 업데이트, (2) 이벤트 핸들러 호출
- Lister: 그 로컬 캐시에서 리소스를 읽는 read-only 인터페이스
 - API 서버에서 읽는게아닌 캐시를 통해 읽어서 빠르고 API 서버 부하 감소
- Workqueue: 이벤트 핸들러가 "이 리소스 처리해야 함"을 큐에 던지면, 워커 고루틴이 꺼내서 reconcile 함수를 호출.
 - 같은 키가 여러 번 들어와도 dedup 되고, 실패 시 rate-limited 재시도



Workqueue는 "처리해야 할 리소스 키(`namespace/name`)들이 쌓여있는 큐
Worker(워커 고루틴)는 그 큐에서 키를 하나씩 꺼내서 `Reconcile` 함수를 호출하는 일을 무한 반복하는 고루틴

* goroutine(고루틴): Go 초경량 스레드

[흐름]

API Server → Watch → Informer → (Cache + Event Handler) → Workqueue → Worker → Reconcile

TODO: 실습 해보기

3주차: Operator SDK & Kubebuilder

학습 목표

- 왜 SDK/Kubebuilder를 쓰는지 (client-go raw 로 쓰면 뭐가 힘든지)
- Scaffolding이 만들어주는 디렉토리 구조의 각 파일이 하는 일
- controller-runtime이 client-go 위에 어떤 추상화를 엮는지

1. 왜 SDK/Kubebuilder가 필요한가

보일러 플레이트

- Informer factory 생성, scheme 등록, GVK(Group/Version/Kind) 매핑
- 여러 리소스를 동시에 watch하는 manager
- Reconcile 함수의 에러/재시도/backoff 처리
- CRD YAML 수동 작성 (필드마다 OpenAPI 스키마, validation, printer columns...)
- Webhook(admission, conversion) 서버 셋업
- 메트릭, 헬스체크, 리더 일렉션
- RBAC YAML 수동 관리

→ Kubebuilder는 이걸 전부 생성해줌

Go struct만 정의하면 CRD YAML이 나오고, Reconcile 함수의 시그니처만 채우면 `workqueue` / `retry` / `cache` 가 자동으로 붙는다.

Operator SDK vs Kubebuilder

둘 다 내부적으로 controller-runtime을 사용

Kubebuilder - 미니멀, 군더더기 없음

Operator SDK - Helm/ Ansible operator 도 지원하고 OLM(Operator Lifecycle Manager) 통합이 강함

Go operator만 만들거면 사실상 둘이 거의 동일

2. Scaffolding 프로젝트 구조 분석

`kubebuilder init` + `kubebuilder create api` 를 돌리면 나오는 구조:

```
.
├── Dockerfile
├── Makefile           # build/test/deploy 타겟 모음
├── PROJECT            # kubebuilder 메타데이터
├── cmd/
│   └── main.go        # Manager 생성, 컨트롤러 등록, 시작점
└── api/
```

```

├─ v1alpha1/
│   └─ mykind_types.go # ← CR 스펙(Go struct) 정의. 가장 중요.
│   └─ groupversion_info.go
│   └─ zz_generated_deepcopy.go # 자동 생성, 손대지 말 것
├─ internal/controller/
│   └─ mykind_controller.go # ← Reconcile 로직. 두 번째로 중요.
│   └─ suite_test.go # envtest 기반 통합 테스트
├─ config/
│   └─ crd/ # 자동 생성된 CRD YAML
│   └─ rbac/ # ServiceAccount, Role, RoleBinding
│   └─ manager/ # Deployment manifest
│   └─ default/ # customize entry point
│   └─ samples/ # CR 예시
└─ hack/boilerplate.go.txt

```

열어봐야할 파일 4개

- `cmd/main.go` — `ctrl.NewManager` → `SetupWithManager` 흐름. Manager가 모든 컨트롤러, 캐시, webhook 서버의 root.
- `api/v1alpha1/mykind_types.go` — `Spec` (desired)과 `Status` (observed) struct. `// +kubebuilder:validation:...` 마커로 OpenAPI 스키마가 생성됨.
- `internal/controller/mykind_controller.go` — `Reconcile(ctx, req)` 함수와 `SetupWithManager`. `For(&mygroupv1.MyKind{}).Owns(&corev1.Pod{}).Complete(r)` 형태로 watch 대상 등록.
- `config/crd/bases/...yaml` — Go struct에서 자동 생성된 CRD. `make manifests` 돌리면 갱신.

3. controller-runtime 라이브러리

controller-runtime: client-go 위에 추상화 레이어

client-go (raw)	controller-runtime
<code>Informer</code> + <code>SharedInformerFactory</code>	<code>Manager</code> 의 내장 cache
<code>Lister</code>	<code>client.Client</code> 의 <code>Get/List</code> (캐시 자동 사용)
<code>Workqueue</code> + work goroutine	<code>Reconciler</code> 인터페이스 + 내부 큐
직접 watch 설정	<code>For()</code> / <code>Owns()</code> / <code>Watches()</code> 빌더
직접 retry 로직	<code>Result{Requeue: true, RequeueAfter: ...}</code> 반환

`Reconcile` 함수 시그니처:

```

func (r *MyKindReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    // req.NamespacedName으로 대상 리소스 식별
    // 1. Get으로 현재 spec 읽기
    // 2. 원하는 상태 계산
    // 3. 자식 리소스(Pod, Deployment 등) 생성/수정/삭제
    // 4. Status 업데이트
    return ctrl.Result{}, nil
}

```

Reconcile의 황금률: 멱등(idempotent)해야 한다. 같은 입력에 몇 번을 호출해도 결과가 같아야 함. 왜냐면 controller-runtime이 임의로 재호출하기 때문 (재시도, 캐시 동기화 후 재처리 등). "Pod를 추가 생성"이 아니라 "Pod가 *있어야 한다*"는 식으로 짜야 합니다.